dependencies that are only needed for a JSON-processing page handler, that suggests a bit of a code smell. At that point, or if your needs are more extensive, you should consider using *minimal request-handling APIs*, a new feature introduced in .NET 6.

11.3 Minimal request-handling APIs

Before .NET 6, the recommended way to handle extensive services over HTTP in ASP.NET Core, such as those that consume and produce JSON, was to use the Web API framework that forms part of the MVC framework. To do that, you create classes that derive from `ApiController`, add the required action methods to them for processing requests, add the relevant services to your application, and map the controller action methods as endpoints. Incorporating Web API controllers in your application involves a degree of ceremony.

If you recall from chapter 1, a larger part of the motivation behind the introduction of Razor Pages was to reduce the ceremony required by the existing MVC framework to generate HTML on the server. The effort to reduce ceremony has continued in .NET 6. You have already seen how application bootstrapping and configuration have been simplified down to one file, in chapter 2, based on a new minimal hosting API. As part of an overall *minimal APIs* feature, a slimmed-down approach to handling requests was also introduced that allows you to map a route to a function, and it *just works*.

You register minimal API request handlers with the `Map[HttpMethod]` method on `WebApplication`—`MapPost`, `MapGet`, `MapPut`, and so on—using the same convention with which you register page handler methods in a `PageModel` class. Recall that an